National University of Computer and Emerging Sciences



Laboratory Manuals

for

Data Structures Lab (CL-2001)

Course Instructor	Mr. Waqas Rana
Lab Instructor	Ms. Fatima Ali
Lab TA	Mr. Abdur Rehman
Section	BSE-3B
Semester	Fall 2026

Department of Computer Science

FAST-NU, Lahore, Pakistan

Lab Manual 05

Objectives:

- Learn how to solve difficult level problems on Queues

Queue

A Queue is a linear data structure that follows the principle of:

FIFO (First In, First Out)

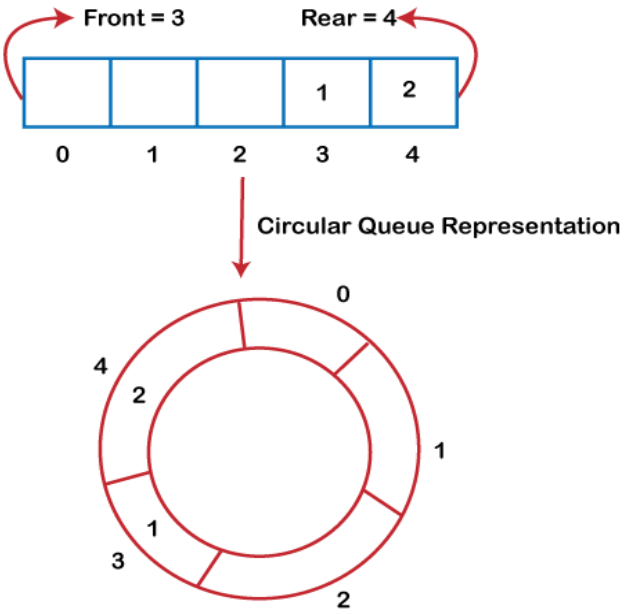
The element inserted first is removed first. Think of it like a line at a ticket counter: the first person in line is the first served.

Structure & Terminology

- Front: points to the element that will be removed first.
- Rear: points to the last inserted element.
- Enqueue: operation to insert at rear.
- Dequeue: operation to remove from front.

Applications of Queue

- Operating Systems: CPU scheduling, Disk scheduling.
- Printers: Print job management.
- Simulations: Traffic, customer service, events.



Lab Tasks

Task 1: Cafeteria Token System (10)

Imagine the FAST-NU cafeteria during the lunch break. Hundreds of students want to order food but there is only one serving counter. To keep things fair, the cafeteria uses a token system: every student who arrives receives a token number, and students are served strictly in the order in which they arrived. Nobody is allowed to jump the line.

- Each new student is issued the next token number automatically (starting from 1) along with their name and food order.
- When the counter is free, the student who has waited the longest (the front of the queue) is called, served and leaves the queue.
- The staff can peek at who is next without serving them.
- Students keep arriving while others are being served, so the queue grows and shrinks throughout the day.

```
==== Cafeteria Token Menu ====
1. Issue New Token
2. Serve Next Student
3. Show Next in Line (Peek)
4. Show Waiting Queue
5. Count Waiting Students
6. Exit
Enter choice: _
```

Your program should simulate the cafeteria using a **Queue**. Provide a menu with the following options:

1. **Issue New Token**
   - Input: student name and food order.
   - Action: generate the token number automatically and **enqueue** the student.
2. **Serve Next Student**
   - Action: **dequeue** the front student and display the token, name and order served.
   - If nobody is waiting, display "No students waiting".
3. **Show Next in Line**
   - Display the front student without removing them (peek).
4. **Show Waiting Queue**
   - Display all waiting students from front to rear.

5. **Count Waiting Students**
    - Display how many students are currently in the queue.
6. **Exit**: Terminate the program safely.

Sample Run

```
Enter choice: 1
Name: Fatima
Order: Pasta
Token #1 issued.
```

```
Enter choice: 2
Serving Token #1 (Fatima): Pasta
```

```
Enter choice: 3
Next in line: Token #2 (Hina)
```

Task 2: Implementation of Queue (5)

Implement a queue using a circular array with operations:

- enqueue() (insert)
- dequeue() (remove)
- peek() (see front element)
- isEmpty() / isFull().
- display() (show all elements from front to rear)
- getSize() (number of elements currently stored)

Note: Use the modulo operator (%) so that front and rear wrap around the array. You will reuse this class in Task 3.

Task 3: The Josephus Problem (Application of Circular Queue) (10)

The **Josephus problem** is a famous theoretical problem, named after the historian Flavius Josephus, and a classic application of a **circular queue**. It can be seen as an elimination game:

- There **are n** people standing in a circle waiting to be eliminated.
- The counting begins at some point in the circle and proceeds around the circle in a **fixed direction**.
- In each step, a certain number of people are **skipped** and the **next person is eliminated**.
- The elimination proceeds around the circle (which is becoming smaller and smaller as the eliminated people are removed) until **only the last person remains**, who is given freedom (the **survivor**).
- Given the total number of persons **n** and a number **k** which indicates that **k-1** persons are skipped and the **k-th** person is eliminated, find the **survivor** and the **order of elimination**.

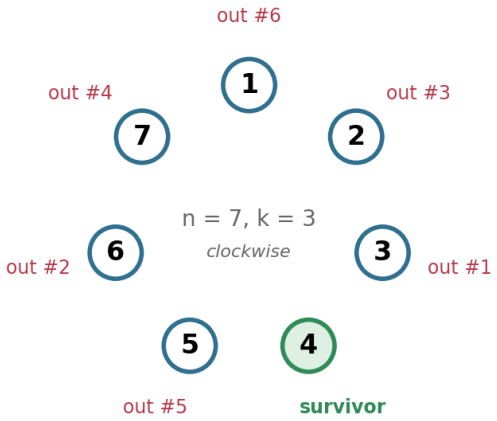


Figure: n = 7, k = 3. Elimination order 3, 6, 2, 7, 5, 1 and the survivor is 4.

Dry Run (n = 7, k = 3)

Round	Queue at start (front first)	Skipped (moved to rear)	Eliminated
1	1 2 3 4 5 6 7	1, 2	3
2	4 5 6 7 1 2	4, 5	6
3	7 1 2 4 5	7, 1	2
4	4 5 7 1	4, 5	7
5	1 4 5	1, 4	5
6	1 4	1, 4	1
End	4	-	**Survivor = 4**

Requirements

1. Take **n** and **k** from the user (both must be at least 1; reject invalid input).
2. Use the **circular array queue** you built in Task 2 (capacity n). Do not shift array elements and do not use the STL queue.
3. Fill the queue with 1 to n, then simulate the eliminations as described above.
4. Print every eliminated person in order, and the queue contents after each round.
5. Print the survivor at the end.

Task 5: Reverse a Queue (5)

Write a C++ program that reverses the elements of a queue.

Input: q[] = [5, 10, 15, 20, 25]
Output: [25, 20, 15, 10, 5]

Task 6: Generate Binary Numbers Using a Queue (5)

Write a C++ program that generates the binary representations of numbers from 1 to n using a queue. The program should generate binary numbers by using the queue rather than converting each decimal number individually.

Input: n = 4
Output: ["1", "10", "11", "100"]

Submission Guidelines

1. submit following the only file strictly following the naming convention
    a. l261234.cpp